

Docker & Docker Compose

Utilité dans le projet

Docker permet de **conteneuriser** l'application, c'est-à-dire d'emballer chaque composant (backend, frontend, MongoDB) avec toutes ses dépendances dans des conteneurs isolés et portables.

Avantages pour notre projet

-  **Portabilité** : L'application fonctionne partout (Windows, Mac, Linux)
-  **Isolation** : Chaque service tourne dans son propre environnement
-  **Reproductibilité** : Même environnement en dev et prod
-  **Rapidité** : Démarrage instantané vs machines virtuelles

Composants du projet

1. Backend (API Node.js)

Dockerfile : `backend/Dockerfile`

```
FROM node:22-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --production
COPY . .
EXPOSE 5000
CMD ["node", "server.js"]
```

Ce qui se passe :

1. Image de base : Node.js 22 Alpine (légère)
2. Installation des dépendances
3. Copie du code source
4. Exposition du port 5000
5. Démarrage du serveur Express

2. Frontend (React + Nginx)

Dockerfile : `frontend/Dockerfile`

```
# Stage 1: Build React
FROM node:22-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Stage 2: Nginx
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

Multi-stage build :

1. **Stage 1** : Compile React avec Vite → génère `/dist`
2. **Stage 2** : Copie seulement les fichiers compilés dans Nginx
3. **Résultat** : Image finale légère (20 MB vs 400 MB)

3. MongoDB

Utilise l'**image officielle** `mongo:7` (pas de Dockerfile custom)



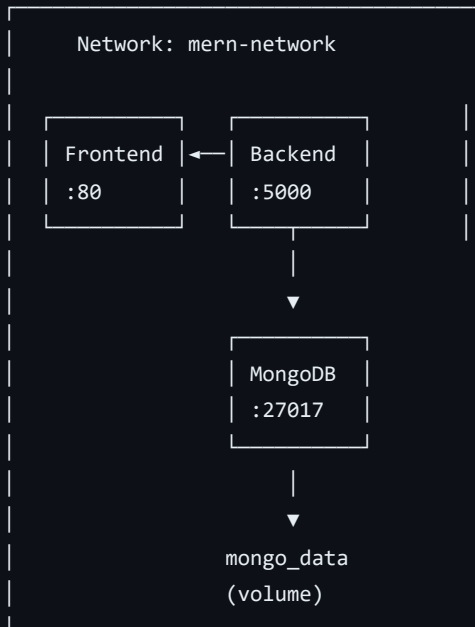
Docker Compose

Fichier : `docker-compose.yml`

Rôle

Orchestrer les 3 services (mongo, backend, frontend) en une seule commande.

Architecture



Configuration clé

```
services:
  mongo:
    image: mongo:7
    volumes:
      - mongo_data:/data/db # Persistence des données
    healthcheck:
      test: ["CMD", "mongosh", "--eval", "db.adminCommand('ping')"]

  backend:
    build: ./backend
    depends_on:
      mongo:
        condition: service_healthy # Attend que MongoDB soit prêt
    environment:
      - MONGO_URI=mongodb://mongo:27017/filrouge

  frontend:
    build: ./frontend
    depends_on:
      backend:
        condition: service_healthy
    ports:
      - "80:80" # Accessible sur http://localhost
```



Commandes essentielles

Démarrage

```
# Build et démarrer tous les services
docker compose up --build

# En arrière-plan
docker compose up -d

# Logs en temps réel
docker compose logs -f backend
```

Arrêt

```
# Arrêter les conteneurs
docker compose stop

# Supprimer conteneurs + volumes
docker compose down -v
```

Debug

```
# Voir les conteneurs actifs
docker compose ps

# Entrer dans un conteneur
docker exec -it backend sh

# Voir les logs
docker compose logs backend

# Rebuild un service
docker compose up --build backend
```

Volumes et persistance

mongo_data

- **Type** : Volume nommé
- **But** : Persister la base de données MongoDB
- **Emplacement** : Géré par Docker (pas dans le projet)

backend/uploads

- **Type** : Bind mount
- **But** : Persister les images uploadées
- **Emplacement** : `./backend/uploads`



mern-network

- **Type** : Bridge
- **But** : Communication inter-conteneurs
- **DNS** : Les services se trouvent par leur nom
 - Backend → `mongodb://mongo:27017`
 - Frontend → `http://backend:5000`



Bonnes pratiques appliquées

- ☒ `.dockerignore` : Exclut `node_modules`, `.env`
- ☒ Multi-stage : Réduit la surface d'attaque
- ☒ Images Alpine : Légères et sécurisées
- ☒ Secrets via `.env` : Pas dans l'image
- ☒ Healthchecks : Détection des défaillances



1. **Docker Swarm** : Orchestration multi-nœuds
2. **Watchtower** : Mise à jour automatique des images
3. **Traefik** : Reverse proxy + HTTPS
4. **Portainer** : Interface web de gestion



Docker est utilisé dans le **Jenkinsfile** :

```
stage('Build & Push') {  
  steps {  
    sh 'docker build -t lims4/backend:latest ./backend'  
    sh 'docker push lims4/backend:latest'  
  }  
}
```

Les images sont poussées sur **Docker Hub** et déployées sur **Kubernetes**.

Prochaine étape : [02-JENKINS.md](#)